

# 벡터 미분 기반 해리스 코너 감지 알고리즘

## 1. 탐구 동기 및 목적

컴퓨터 비전 분야에서 경계나 코너를 감지하기 위해 사용하는 다양한 알고리즘 중 해리스 코너 감지 알고리즘을 접하게 되었다. 처음 이 알고리즘을 접했을 때 이 알고리즘이 그레이스케일 이미지만을 대상으로 설계되었다는 사실을 알게 되었다. 하지만 실제로 마주하게 되는 대부분의 이미지는 그레이스케일이 아닌 RGB 3가지 색상 정보로 이루어져 있기 때문에 이를 그레이스케일로 변환하는 과정에서 발생하는 정보 손실이 생긴다는 점을 알게 되었다.

예를 들어 빨간 물체와 초록 물체가 서로 맞닿은 경계는 밝기는 거의 같을 수 있지만 색은 다르기에 감지가 되어야 한다. 하지만 이러한 경우 그레이스케일 변환 과정에서 색 정보가 압축되어 이러한 경계를 놓칠 가능성이 존재한다. 이에 따라 이미지 내에서 경계나 코너를 구분하는 과정에서 미분을 활용하는 이유와 이를 활용한 알고리즘의 원리를 탐구해보고, 원본 이미지의 색 정보를 버리지 않고 RGB 이미지를 벡터장의 형태로 모델링하여 기존 알고리즘을 확장해보려 한다.

## 2. 관련 수학적 개념 및 이론적 배경

### 1) 그레이스케일 이미지

그레이스케일 이미지란 색 정보를 모두 제거하고 검은색부터 흰색까지의 밝기 단계만을 사용하여 표현되는 흑백의 이미지이다. 즉 이미지를 함수의 형태로 모델링한다면 함수  $f(x, y)$ 의 결과값은 하나의 실수 형태이다.

### 2) 그래디언트와 편미분

여러 변수로 이루어진 함수의 경우 미분을 수행하는 변수에 따라 결과가 달라지게 된다. 이때 다변수 함수에서 특정 변수만을 변화하는 변수로 취급하고 나머지 변수들을 고정된 상수로 취급하여 미분하는 것을 편미분이라고 한다.

다변수 함수에서는 각 변수에 대해 편미분한 결과를 이용하여 그래디언트라는 벡터를 만들 수 있는데, 이는 함수 내 해당 지점에서 함수값이 가장 빠르게 증가하는 방향을 가르킨다.

$$\nabla f = \left( \frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right)$$

그래디언트의 크기는 밝기 변화의 강도를 나타내고, 방향은 밝기가 가장 빠르게 증가하는 방향을 나타낸다. 하지만 이론적 정의와 달리 실제 이미지는 연속적인 색상값을 가지지 않기 때문에 편미분의 값을 아래와 같이 근사를 통해 구해야 한다.

$$\frac{\partial f}{\partial x} \approx \frac{f(x+1, y) - f(x-1, y)}{2}$$

### 3) 가우시안 함수와 합성곱

이미지에는 기본적으로 노이즈가 포함되어 있다. 그렇기 때문에 겉으로 보았을 때에는 부드러운 색상 변화이더라도 픽셀 단위의 미세한 값 변화에서는 노이즈가 매우 큰 영향을 미치게 된다. 그렇기에 그래디언트를 근사를 통해 수치적으로 계산하게 되면 노이즈로 인해 실제 값보다 절대적인 값이 증폭되어 계산된다. 이를 해결하기 위한 기법이 2차원 가우시안 함수를 이용한 스무딩이다. [1] 이는 이미지 내의 픽셀에 대해 가우시안 커널로 합성곱 연산을 수행하여 평균을 내는 방식으로 동작한다.

2차원으로 확장한 가우시안 함수는 다음과 같다.

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2 + y^2}{2\sigma^2}}$$

이 함수를 이용해 가우시안 커널을 만들게 되면

|   |    |    |    |   |
|---|----|----|----|---|
| 1 | 3  | 7  | 4  | 1 |
| 4 | 16 | 26 | 16 | 4 |
| 7 | 26 | 41 | 26 | 7 |
| 4 | 16 | 26 | 16 | 4 |
| 1 | 4  | 7  | 4  | 1 |

$$\times \frac{1}{273}$$

가 나온다.

가우시안 커널의 값을 전부 더하게 되면 1이 된다. 따라서 커널로 합성곱을 진행하게 되더라도 색상들의 분포가 전부 커지거나 작아지는 일은 발생하지 않는다. 단지 가우시안 분포에 따라 값들이 부드러워지며 노이즈에 의해 발생했던 미분값 증폭이 해결된다.

### 4) 야코비안 행렬

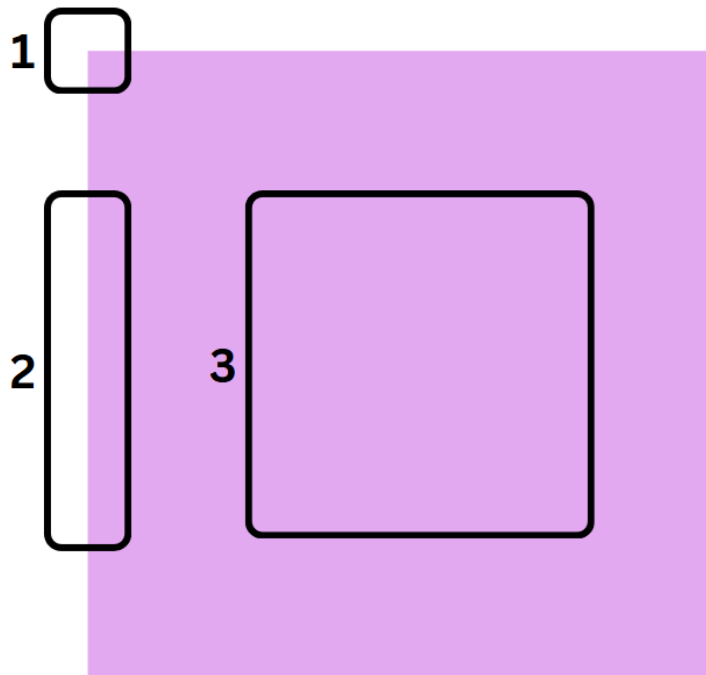
야코비안 행렬은  $n$ 차원 벡터를 입력으로 받고  $m$ 차원 벡터를 출력하는 다변수 벡터함수에 대한 편미분들을 모아놓은 행렬로, 야코비안 행렬의 원소들은 1차 미분 계수로 구성되어 있다. [3]

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$

### 3. 탐구 내용

#### (1) 해리스 코너 검출기

이미지에서 코너는 두 방향 이상으로 밝기가 급격히 변하는 지점으로 정의할 수 있다. 이미지 내에서 영역을 아래와 같이 3가지로 나누어 볼 수 있는데



1번 영역은 코너, 2번 영역은 에지, 3번 영역은 평탄한 영역이다. 1번 영역에서는 어느 방향으로 이동해도 밝기가 급격하게 변화한다. x축 방향으로 이동해도 전환되는 구간이 존재하고, y축 방향으로 이동해도 존재하기 때문이다. 2번 영역은 한 방향으로만 밝기가 급격히 변화하고, 그 경계선을 따라가는 방향으로 이동하면 변화가 없다. 이는 x축 방향으로 색이 전환되는 구간이 존재하지만, y축 방향으로 그렇지 않기 때문이다. 3번 영역에서는 어느 방향으로 이동하더라도 밝기의 변화가 존재하지 않는다. 이러한 차이를 이용해 해당 픽셀에서 조금 이동했을 때의 픽셀의 밝기의 변화를 수치화한다면 코너를 감지할 수 있을 것이다.

이때 단순히 단일 픽셀의 변화를 보는 것이 아닌, 작은 window를 설정하여 그 window를 이동시켰을 때 그 window 내의 픽셀의 변화를 전부 합하는 방식으로 코너를 감지하는 알고리즘이

해리스 코너 감지 알고리즘이다. 하지만 해리스 코너 감지는 그레이스케일 이미지에서만 적용 가능하기 때문에 색상 정보의 손실로 인해 정밀도나 감지 정확성이 저하될 수 있다고 생각하여 이를 RGB 색상 정보를 모두 각각 고려할 수 있는 형태로 확장하게 되었다.

## (2) 이미지 픽셀 정보를 벡터로 확장

이미지에는 RGB 3가지 색상 정보가 각 픽셀마다 존재한다. 따라서 각 픽셀 위치에 따른 색상에 관한 함수  $F(x, y)$ 를 다음과 같이  $x, y$  픽셀 위치에 따른 RGB 각각에 관한 정보가 들어간 벡터 형태로 정의하였다.

$$F(x, y) = \begin{bmatrix} R(x, y) \\ G(x, y) \\ B(x, y) \end{bmatrix}$$

이를 통해 RGB 색상 정보를 손실 없이 사용할 수 있다.

$F(x, y)$ 는 2차원 벡터에 대해 3차원 벡터를 대응시키는 함수이기에 야코비안 행렬은

$$J_F = \begin{bmatrix} \partial_x R & \partial_y R \\ \partial_x G & \partial_y G \\ \partial_x B & \partial_y B \end{bmatrix}$$

가 된다. 이는 각각  $x, y$ 방향에 대해 색상 RGB의 값이 각각 변화하는 정도를 나타낸다.

야코비안 행렬을 방향 벡터와 곱하게 되면 해당 방향으로 이동하였을 때의 색상 변화 벡터를 구할 수 있다.

방향 벡터를

$$v = \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

라 하면

$$D_v F = J_F v = \begin{bmatrix} \partial_x R & \partial_y R \\ \partial_x G & \partial_y G \\ \partial_x B & \partial_y B \end{bmatrix} \begin{bmatrix} v_x \\ v_y \end{bmatrix} = \begin{bmatrix} \partial_x R v_x + \partial_y R v_y \\ \partial_x G v_x + \partial_y G v_y \\ \partial_x B v_x + \partial_y B v_y \end{bmatrix}$$

의 형태로 색상 변화 벡터가 계산된다.

다만 이는 단일 픽셀에 대한 결과이고, 해리스 알고리즘에서는 코너 판별 과정에서 단일 픽셀이 아니라 그 점 주변의 window의 변화를 확인하기 때문에 창 내의 모든 픽셀에 대한 변화량을 합산해야 한다.

$$\begin{aligned}
& E(\Delta x, \Delta y) \\
&= \sum_{(u,v) \in W} w(u,v) [(R(u + \Delta x, v + \Delta y) - R(u,v))^2 \\
&+ (G(u + \Delta x, v + \Delta y) - G(u,v))^2 \\
&+ (B(u + \Delta x, v + \Delta y) - B(u,v))^2]
\end{aligned}$$

변화량이 RGB에 대한 벡터의 형태로 나타나기 때문에 각 성분을 제공하고 더하여 각 픽셀의 차이를 모두 더하였다. 이를 통해 기존 그레이스케일에서와 달리 각 성분의 변화를 개별적으로 계산하여 밝기 뿐만 아니라 색상 변화에 의한 차이도 포함될 것이다.

실제 이미지는 이산적인 픽셀 위치에 대한 색상값을 가지기 때문에 완벽한 편미분 값이 아니라 근사값을 사용해야 한다.

$$\begin{aligned}
\frac{\partial R}{\partial x}(u,v) &\approx \frac{R(u+1,v) - R(u-1,v)}{2} \\
\frac{\partial R}{\partial y}(u,v) &\approx \frac{R(u,v+1) - R(u,v-1)}{2}
\end{aligned}$$

따라서 픽셀값의 차를 이용하여 근사해주었다.

이것을 앞선 식에서 등장하는  $R(u + \Delta x, v + \Delta y) - R(u,v)$ 에 대입하면

$$R(u + \Delta x, v + \Delta y) - R(u,v) \approx \frac{\partial R}{\partial x}(u,v) \cdot \Delta x + \frac{\partial R}{\partial y}(u,v) \cdot \Delta y$$

알고리즘에 따라 window가  $(\Delta x, \Delta y)$ 만큼 이동했다고 한다면

$$\begin{aligned}
E(\Delta x, \Delta y) \approx \sum_{(u,v) \in W} w(u,v) & \left[ \left( \frac{\partial R}{\partial x}(u,v) \cdot \Delta x + \frac{\partial R}{\partial y}(u,v) \cdot \Delta y \right)^2 \right. \\
& + \left( \frac{\partial G}{\partial x}(u,v) \cdot \Delta x + \frac{\partial G}{\partial y}(u,v) \cdot \Delta y \right)^2 \\
& \left. + \left( \frac{\partial B}{\partial x}(u,v) \cdot \Delta x + \frac{\partial B}{\partial y}(u,v) \cdot \Delta y \right)^2 \right]
\end{aligned}$$

식을 전개한 후 정리하면 행렬의 형태로 표현할 수 있다. [2]

$$\frac{\partial R}{\partial x}(u,v) = R_x$$

$$\frac{\partial R}{\partial y}(u,v) = R_y$$

$$\frac{\partial G}{\partial x}(u,v) = G_x$$

$$\frac{\partial G}{\partial y}(u,v) = G_y$$

$$\frac{\partial B}{\partial x}(u,v) = B_x$$

$$\frac{\partial B}{\partial y}(u,v) = B_y$$

$$E(\Delta x, \Delta y)$$

$$\approx \sum_{(u,v) \in W} w(u,v) \begin{bmatrix} \Delta x & \Delta y \end{bmatrix} \begin{bmatrix} R_x^2 + G_x^2 + B_x^2 & R_x R_y + G_x G_y + B_x B_y \\ R_x R_y + G_x G_y + B_x B_y & R_y^2 + G_y^2 + B_y^2 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

이 식이 RGB 색상을 벡터 형태로 모두 고려했을 때의 해리스 코너 검출기의 수식이다. window W를 이동시키면서 연산을 수행하면 코너에서 값이 주변 대비 크게 나타나면서 코너를 검출할 수 있다.

## (2) 알고리즘 구현

유도한 식을 바탕으로 python 코드를 작성하였다.

```
import numpy as np
import cv2

def compute_gradients(channel):
    C = channel.astype(np.float64)
    Cx = (np.roll(C, -1, axis=1) - np.roll(C, 1, axis=1)) / 2.0
    Cy = (np.roll(C, -1, axis=0) - np.roll(C, 1, axis=0)) / 2.0
    return Cx, Cy

def compute_M_RGB(img_rgb, sigma=1.5, window_size=5):
    blurred = cv2.GaussianBlur(img_rgb, (0, 0), sigma)
    R = blurred[:, :, 0]
    G = blurred[:, :, 1]
    B = blurred[:, :, 2]
```

```

Rx, Ry = compute_gradients(R)
Gx, Gy = compute_gradients(G)
Bx, By = compute_gradients(B)

A = Rx**2 + Gx**2 + Bx**2
B_ = Rx*Ry + Gx*Gy + Bx*By
C = Ry**2 + Gy**2 + By**2

k = cv2.getGaussianKernel(window_size, sigma)
kernel = k @k.T
M11 = cv2.filter2D(A, -1, kernel)
M12 = cv2.filter2D(B_, -1, kernel)
M22 = cv2.filter2D(C, -1, kernel)

return M11, M12, M22

def compute_R(M11, M12, M22, k=0.05):
    det = M11 * M22 - M12**2
    trace = M11 + M22
    return det - k * trace**2

def detect_corners(R, threshold_ratio=0.03, nms_size=7):
    threshold = threshold_ratio * R.max()
    R_max = cv2.dilate(R, np.ones((nms_size, nms_size), np.uint8))
    return (R == R_max) & (R > threshold)

def rgb_harris(img_rgb, sigma=1.5, window_size=5, k=0.05,
               threshold_ratio=0.03, nms_size=7):
    M11, M12, M22 = compute_M_RGB(img_rgb, sigma, window_size)
    R = compute_R(M11, M12, M22, k)
    corners = detect_corners(R, threshold_ratio, nms_size)
    return corners, R

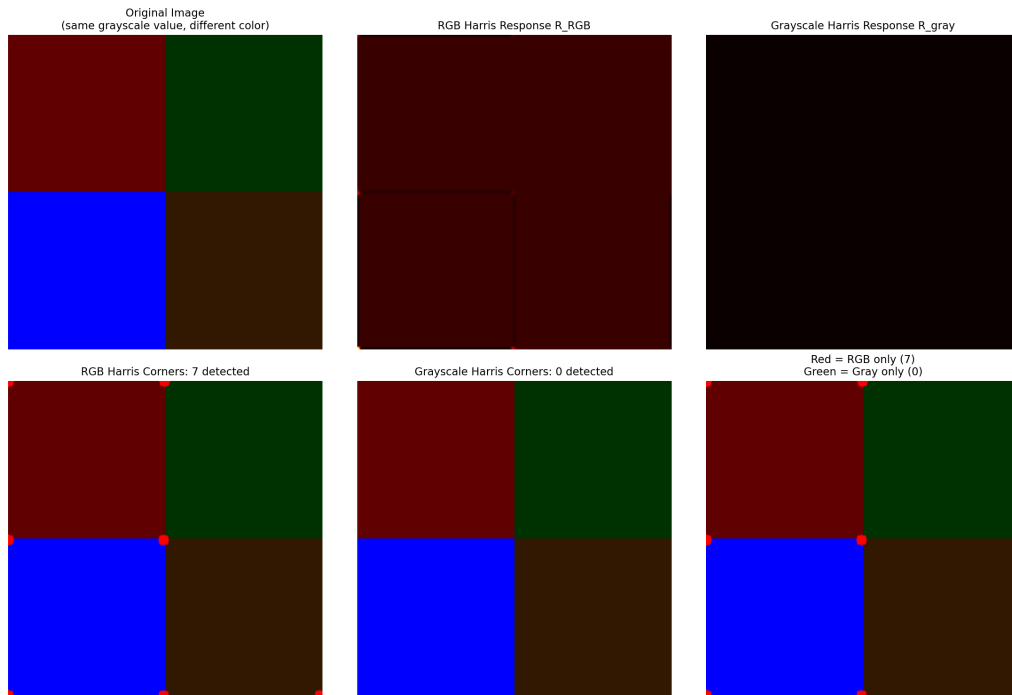
```

<https://gist.github.com/airhood/beed9c873be28fa9ea218cd4c348e3f9>

### (3) 알고리즘 비교 및 분석

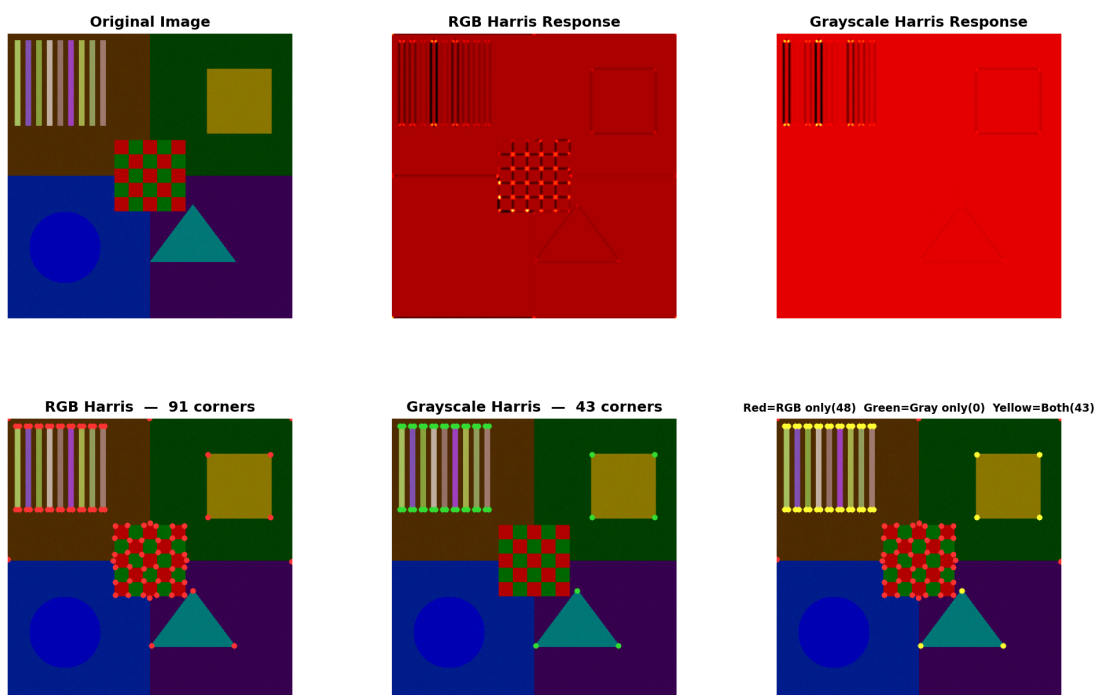
비교를 위해 여러 이미지들을 대상으로 알고리즘을 실행해보았다.

우선 가장 먼저 벡터로 확장한 수식이 그레이스케일 방식 대비 가지는 정밀도를 확인하기 위해 그레이스케일에서는 비슷하게 보이도록 이미지를 생성하고, 이에 대해 코너 감지를 수행해보았다.



그 결과 RGB 색상 정보가 하나의 그레이스케일로 압축되는 과정에서 정보가 손실되는 기존 방식과 다르게 RGB를 각각 따로 취급하는 벡터를 활용한 방식에서 더 나은 성능을 보이는 것을 확인하였다. 위 이미지에서는 기존 방식은 코너를 0개, 벡터를 활용한 새로운 방식이 7개를 검출하였다.

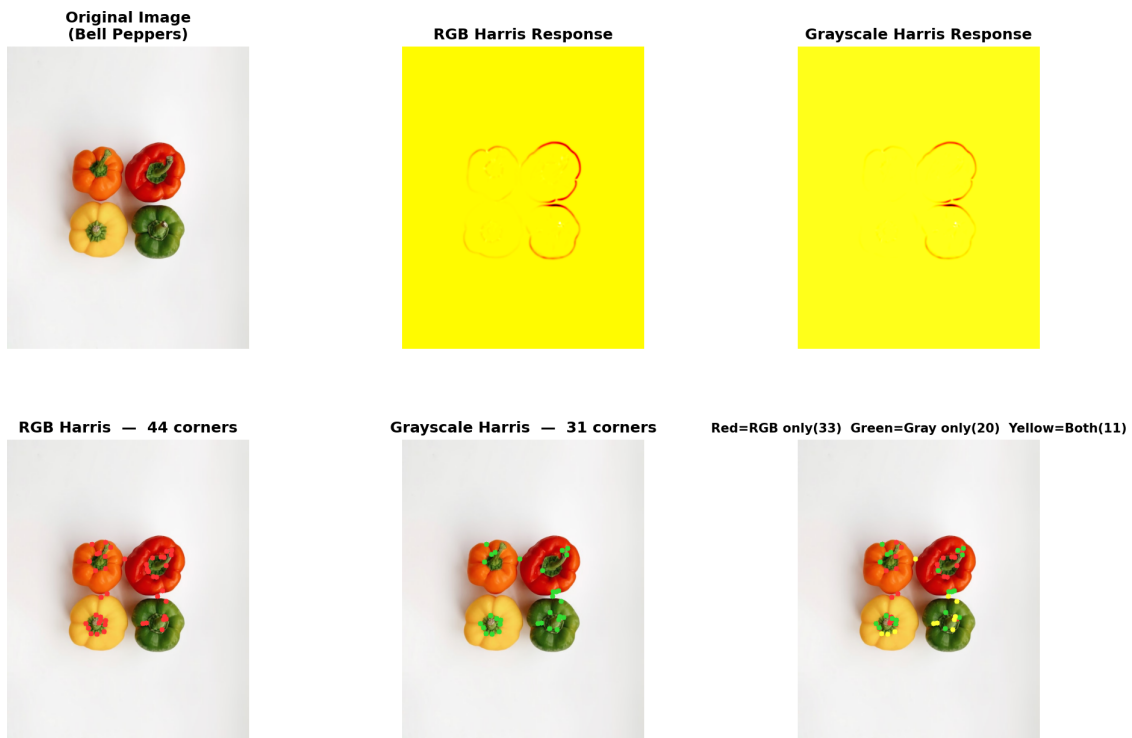
조금 더 다양한 패턴에 대해서도 수행해보았다. 단순히 직각 코너 뿐만 아니라 삼각형이나 격자판 또한 잘 감지하는지 확인하고, 반대로 원과 같이 감지가 되지 않아야 하는 대상은 검출 결과에 포함되지 않는지를 검증하기 위해 아래와 같은 이미지로 코너 감지를 수행해보았다.





이 경우에는 기존 방식 또한 어느정도의 코너를 감지하였지만, 중앙에 배치된 격자판에서는 감지하지 못하는 모습을 보였다. 반면 벡터를 활용한 방식에서는 격자판 내의 작은 코너들을 전부 감지한 것을 확인할 수 있었다. 또한 두 방식 모두 원과 같이 감지를 해서는 안되는 요소들에 대해서는 잘 대응하는 모습을 보였다. 이는 다른 경계 감지 알고리즘들과 다르게 단순히 단일 픽셀에서의 미분값을 통해 단일 픽셀 색상 값의 변화량만을 보는 것이 아니라 window 내의 픽셀들의 미분값의 합을 사용하여 감지를 수행하기에 선을 가로지르는 x 또는 y방향으로의 색 변화만 존재하는 '경계'와 x, y 모두에서 색 변화가 존재하는 '코너'를 구분하기 때문이다.

하지만 앞에서 진행했던 예시들은 이론적으로 생성한 이미지였기 때문에 이것이 현실 이미지에서도 적용될 수 있는지를 확인하기 위해 실제 이미지에 대해서도 코너 감지를 수행해보았다. 다만 이 경우에는 앞선 이미지 예시들에서와 다르게 이미지 속에 노이즈가 존재하기 때문에 '가우시안 스무딩'을 적용한 후 코너 감지를 수행하였다.



실제 이미지의 결과에서도 벡터를 활용한 방식이 기존 방식 대비 경계나 코너가 더 잘 감지되어 나타나는 것을 RGB Harris Response와 Greyscale Harris Response를 통해 확인할 수 있었고, 이것이 더 정밀한 코너 감지로 이어진다는 것을 RGB 벡터를 활용한 방식이 44개, 기존 Greyscale 방식이 31개의 코너를 감지했다는 것을 통해 알 수 있었다.

#### 4. 결론 및 느낀 점

이 탐구에서는 이미지 내 픽셀 색상의 변화를 미분을 통해 계산하여 경계나 코너를 검출하는 방법에

대해 분석하였고, 이러한 아이디어를 적용한 기존 해리스 코너 검출 알고리즘이 그레이스케일에서만 동작한다는 한계점을 개선하기 위해 알고리즘을 그레이스케일이 아닌 RGB 색상 값에 대한 것으로 확장하였다. 확장 과정에서 이미지의 픽셀 색상을 단일 실수값이 아닌 RGB에 관한 벡터의 형태로 표현하였고, 이를 바탕으로 식을 유도하였다. 확장된 식을 통해 이미지가 알고리즘 적용을 위해 그레이스케일로 변환되는 과정에서 손실되는 색상 정보로 인한 정밀도 저하나 오차를 줄일 수 있었고, 확장된 알고리즘을 python 코드로 구현하여 실제 이미지에 적용하였을 때, 실제 이미지 내에서 기존 방식 대비 코너 검출 성능이 향상됨을 확인함으로써 유도한 식과 알고리즘의 성능을 검증하였다.

탐구 과정에서 이미지를 RGB로 이루어진 각 픽셀의 색상에 관한 벡터장으로 모델링하면서 기존에 하나의 변수에 대해서만 나타나는 함수만을 미분했던 것과 달리 다변수 함수나 다변수 벡터 함수를 편미분하는 방법에 대해 배울 수 있었다. 또한 수식 유도 과정에서 수학적인 수식 전개상으로는 연속적인 함수의 미분이 들어간 형태로 나오지만, 실제 이미지 내의 픽셀은 간격이 이산적이라는 문제를 인식하고, 이를 유한 차분으로 근사하여 해결하였다.

수식 유도 과정에서 가장 어려웠던 부분은 이미지를 벡터장으로 모델링하게 되면서 기존 스칼라 함수의 미분을 벡터장으로 확장하는 과정이었다. 그레이스케일 이미지에서는 함수의 출력이 벡터가 아닌 스칼라이기 때문에 입력  $x, y$ 에 대해 편미분을 진행하여 그래디언트를 구성할 수 있었지만, RGB를 벡터로 묶게 되면서 미분의 결과가 야코비안 행렬의 형태가 나오게 되었고, 미분의 결과가 행렬이라는 점이 처음에는 직관적으로 와닿지 않았다. 이를 극복하기 위해 스칼라에서의 수식을 RGB 각각에 대해 따로 전개해본 뒤 세 결과를 하나의 행렬로 묶으면 야코비안이 된다는 것을 확인해봄으로써 이해를 높일 수 있었다.

#### ※ 참고 문헌 및 출처

- [1] Marr, D. & Hildreth, E. (1980). Theory of Edge Detection. Proceedings of the Royal Society of London B, 207, 187-217.
- [2] Harris, C. & Stephens, M. (1988). A Combined Corner and Edge Detector. Proceedings of the 4th Alvey Vision Conference, Manchester, pp. 147-151.
- [3] 양 원영 (2020). 자코비안(Jacobian) 행렬의 기하학적 의미. 공돌이의 수학정리노트. <https://angeloyeo.github.io/2020/07/24/Jacobian.html>